

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-a527a2e4154608d3f.jsonl`  
- SHA-256: `24ad989128b4d372648319b1136750582d7d5be7df990acb6fbb2f445cd35b04`  
- Source modified: `2026-06-18T07:54:35+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_e5140909-890`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T07:52:17.055Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch, unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix + confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.

CANDIDATE:

```
{  
  "title": "Per-clip begin/end padding makes adjacent rallies' clips overlap, replaying footage  
the merge step promises never to duplicate",  
  "file": "backend/pipeline/stitcher/compiler.py",  
  "line": "268-269 (padding applied per-segment after _merge_overlapping at 201)",  
  "trigger": "Two genuinely distinct rallies with a RAW gap smaller than  
begin_padding+end_padding (default 0.5+0.5=1.0s). _merge_overlapping only collapses ranges that  
overlap/abut on raw boundaries, so e.g. rallies (10.0,15.0) and (15.6,20.0) survive as two  
clips. The first is trimmed to padded_end=15.0+0.5=15.5 and the second to  
padded_start=15.6-0.5=15.1, so frames 15.1s-15.5s are emitted in BOTH clips and play twice in  
the reel.",  
  "why_wrong": "_merge_overlapping's docstring and module comment assert 'A highlight reel must  
never replay the same footage.' but that invariant is enforced only on UNPADDED ranges; padding  
is then added independently per clip afterward. Any two rallies closer than the combined padding  
produce overlapping output, violating the stated contract (duplicated frames at the clip  
seam).",  
  "severity": "low",  
  "proposed_fix": "Apply begin/end padding to each (start,end) BEFORE the merge/overlap collapse  
(or run a second merge pass on the padded ranges) so overlapping padded ranges are coalesced  
into one clip instead of two overlapping clips.",  
  "confidence": 0.5,  
  "area": "backend/pipeline/segmenters/ + stitcher/ + validators/"  
}
```

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/pipeline/stitcher/compiler.py + related test. Two lenses below; you are:
REACHABILITY ? prove a real path hits it or set real=false.

2. assistant (2026-06-18T07:52:19.444Z)

I'll analyze the candidate bug about per-clip padding making adjacent rallies' clips overlap.
Let me read the actual code first.

3. user (2026-06-18T07:52:20.323Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)
26
27        @staticmethod
28        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29            if watermark is None:
30                return None
31            if hasattr(watermark, "model_dump"):
32                return watermark.model_dump()
33            if isinstance(watermark, dict):
34                return dict(watermark)
35            return None
36
37        @staticmethod
38        def _ff_quote(path: str) -> str:
39            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41            slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44            option parser treats ':' as the option separator, so a Windows drive-letter
45            path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46            the encode fails with "Invalid argument". This silently disabled the
47            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48            return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50        @staticmethod
51        def _parse_time(val: Union[int, float, str]) -> float:
52            """Normalizes a timestamp value to float seconds.
53            Handles: float/int (pass-through), plain numeric strings ("12.4"),
54            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55            This mirrors the parse_time logic used in gemini.py to ensure
56            the stitcher can always consume whatever format the indexer produced."""
57            if isinstance(val, (int, float)):
58                return float(val)
59            if isinstance(val, str):
60                val = val.strip()

```

```

61         if ":" in val:
62             # Handle MM:SS, HH:MM:SS, etc.
63             parts = val.split(":")
64             parts.reverse()
65             total = 0.0
66             for i, p in enumerate(parts):
67                 try:
68                     total += float(p) * (60 ** i)
69                 except ValueError:
70                     pass
71             return total
72         try:
73             return float(val)
74         except ValueError:
75             logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76         return 0.0
77         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78         return 0.0
79
80     @staticmethod
81     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84         A highlight reel must never replay the same footage. The same rally can land in
85         the segment list more than once ? e.g. two detector runs accumulated for one
86         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
87         or
88         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
89         rally
90         twice. Merging any range that overlaps or abuts the previous one collapses true
91         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93         """
94         parsed = []
95         for raw_s, raw_e in segments:
96             s = VideoCompiler._parse_time(raw_s)
97             e = VideoCompiler._parse_time(raw_e)
98             if e > s:
99                 parsed.append((s, e))
100         parsed.sort()
101         merged: List[Tuple[float, float]] = []
102         for s, e in parsed:
103             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105             else:
106                 merged.append((s, e))
107         return merged
108
109     def _get_video_duration(self, video_path: str) -> float:
110         """Probes the video file to get its total duration in seconds.
111         Returns 0.0 if it cannot be determined."""
112         try:
113             result = subprocess.run(
114                 [
115                     "ffprobe", "-v", "error",
116                     "-show_entries", "format=duration",
117                     "-of", "default=noprint_wrappers=1:nokey=1",
118                     video_path
119                 ],
120                 capture_output=True, text=True, check=True
121             )
122             return float(result.stdout.strip())

```

```

121         except Exception as e:
122             logger.warning(f"Could not determine video duration via ffprobe: {e}")
123             return 0.0
124
125     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129         because the mark is already baked into every clip identically."""
130         wm = self.watermark
131         if not wm or not wm.get("enabled"):
132             return None
133
134         text = (wm.get("text") or "").strip()
135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
138             logo_path = ""
139         if not text and not logo_path:
140             return None
141
142         margin = int(wm.get("margin", 24))
143         opacity = float(wm.get("opacity", 0.85))
144         position = str(wm.get("position", "bottom-right"))
145         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146         dt_xy = {
147             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148             "bottom-left": f"x={margin}:y=h-th-{margin}",
149             "top-right": f"x=w-tw-{margin}:y={margin}",
150             "top-left": f"x={margin}:y={margin}",
151         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152         ov_xy = {
153             "bottom-right": f"W-w-{margin}:H-h-{margin}",
154             "bottom-left": f"{margin}:H-h-{margin}",
155             "top-right": f"W-w-{margin}:{margin}",
156             "top-left": f"{margin}:{margin}",
157         }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159         drawtext = ""
160         if text:
161             ratio = float(wm.get("font_size_ratio") or 0.035)
162             divisor = max(1, round(1.0 / ratio))
163             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
164             tf_path = os.path.join(tmp_dir, "watermark.txt")
165             with open(tf_path, "w", encoding="utf-8") as fh:
166                 fh.write(text)
167             # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168             # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
169             font_path = (wm.get("font_path") or "").strip()
170             if not font_path and _BUNDLED_FONT.exists():
171                 font_path = str(_BUNDLED_FONT)
172             if font_path:
173                 font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174             else:
175                 font_opt = f"font='{wm.get('font', 'Sans')}'"
176             box = ""
177             if wm.get("box", True):
178                 box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179             drawtext = (
180                 f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181                 f"f:fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"

```

```

182         )
183
184         if logo_path and drawtext:
185             return
186         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187         if logo_path:
188             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189         return drawtext
190
191     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
192 float]], output_filename: str) -> str:
193         """
194         Extracts selected segments from a raw video and stitches them together
195         into a seamless highlights file, applying end_padding to prevent abrupt endings.
196         """
197         if not segments:
198             raise ValueError("No highlight segments provided to compile.")
199
200         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
201         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
202         original_count = len(segments)
203         segments = self._merge_overlapping(segments)
204         if len(segments) < original_count:
205             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
206 -> %d.",
207                             original_count - len(segments), original_count, len(segments))
208         if not segments:
209             raise ValueError("No valid (start < end) highlight segments after merge.")
210
211         if not os.path.exists(raw_video_path):
212             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
213 {raw_video_path}")
214
215         # Defense-in-depth: never let the output name escape output_dir (the API also
216         # validates this at the request boundary).
217         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
218
219         # Probe video duration so we can detect out-of-range segments
220         video_duration = self._get_video_duration(raw_video_path)
221         if video_duration > 0:
222             logger.info(f"Source video duration: {video_duration:.2f}s")
223         else:
224             logger.warning("Video duration unknown. Cannot validate segment boundaries
225 against video length.")
226
227         # We will create temporary clips and stitch them using FFmpeg concat demuxer
228         temp_clips = []
229         skipped_segments = []
230
231         # Create a temp directory within output_dir
232         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
233             logger.info(f"Trimming {len(segments)} segments
234 (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
235             # Build the (optional) watermark filtergraph once; it is applied during
236             every
237             # per-clip re-encode so the mark is baked into each clip identically.
238             watermark_filter = self._watermark_filter(tmp_dir)
239             if watermark_filter:
240                 logger.info("[watermark] applying branding overlay to each clip.")
241             for idx, (raw_start, raw_end) in enumerate(segments):
242                 # Normalize timestamps to float seconds ? handles float, int,
243                 # plain numeric strings, and colon-separated formats like "MM:SS"
244                 start = self._parse_time(raw_start)
245                 end = self._parse_time(raw_end)
246                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -

```

```

{end:.2f}s]"
240
241         # Validate the segment times
242         if start < 0:
243             logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
244             start = 0.0
245
246         if start >= end:
247             logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
248             skipped_segments.append((idx, start, end, "start >= end"))
249             continue
250
251         # Check if segment extends beyond video duration
252         if video_duration > 0:
253             if start >= video_duration:
254                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
255                             f"This segment cannot be extracted ? the video ran
out. Skipping.")
256                 skipped_segments.append((idx, start, end, "start beyond video
end"))
257                 continue
258
259             if end > video_duration:
260                 original_end = end
261                 end = video_duration
262                 logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
263                             f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
264
265                 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
266
267                 # Precise sub-second trim using fast re-encoding to preserve stream
headers
268                 padded_start = max(0.0, start - self.begin_padding)
269                 padded_end = end + self.end_padding
270
271                 # Clamp padded_end to video duration if known
272                 if video_duration > 0 and padded_end > video_duration:
273                     padded_end = video_duration
274                     logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
275                             f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
276
277                 trim_duration = padded_end - padded_start
278                 logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
279
280                 base_cmd = [
281                     "ffmpeg", "-y",
282                     "-ss", str(round(padded_start, 3)),
283                     "-to", str(round(padded_end, 3)),
284                     "-i", raw_video_path,
285                     "-c:v", "libx264",
286                     "-preset", "superfast",
287                     "-crf", "22",
288                     "-c:a", "aac",
289                     "-b:a", "128k",
290                 ]
291                 vf_args = ["-vf", watermark_filter] if watermark_filter else []
292

```

```

293         # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
294         # path), retry once WITHOUT it so a branding misconfig can never nuke
the
295         # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
296         attempts = [vf_args, []] if vf_args else [[]]
297         produced = False
298         last_exc: Optional[subprocess.CalledProcessError] = None
299         cmd = base_cmd + [clip_path]
300         for attempt_vf in attempts:
301             cmd = base_cmd + attempt_vf + [clip_path]
302             try:
303                 subprocess.run(cmd, capture_output=True, check=True)
304             except subprocess.CalledProcessError as e:
305                 last_exc = e
306                 if attempt_vf and vf_args:
307                     _err = (e.stderr.decode(errors="replace").strip()[-300:]
308                             if getattr(e, "stderr", None) else "")
309                     logger.warning(
310                         f"{segment_label}: watermark encode failed; retrying
without it. "
311                         f"ffmpeg: {_err or '(no stderr captured)'}"
312                     )
313                     continue
314                     break
315                 if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
316                     produced = True
317                 if not attempt_vf and vf_args:
318                     logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")
319                     break
320                 last_exc = None # ffmpeg succeeded but produced nothing
321                 break
322
323         if produced:
324             temp_clips.append(clip_path)
325         elif last_exc is not None:
326             stderr_msg = last_exc.stderr.decode(errors='replace').strip()
327             logger.error(f"{segment_label}: FFmpeg trim failed.")
328             print(f"  Command: {' '.join(cmd)}")
329             print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
330             skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
331             continue
332         else:
333             logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
334             skipped_segments.append((idx, start, end, "empty output clip"))
335             continue
336
337         # Report summary of skipped segments
338         if skipped_segments:
339             logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
340             for seg_idx, seg_start, seg_end, reason in skipped_segments:
341                 logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
342             logger.info("")
343
344         if not temp_clips:
345             raise RuntimeError(
346                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
347                 f"All {len(segments)} segments were skipped. "

```

```

348             f"Check the errors above for details. "
349             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
350         )
351
352         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
353
354         # Create a list file for FFmpeg concat demuxer
355         list_file_path = os.path.join(tmp_dir, "clips.txt")
356         with open(list_file_path, "w") as f:
357             for clip in temp_clips:
358                 # Escape single quotes in path
359                 safe_path = clip.replace("'", "'\\'")
360                 f.write(f"file '{safe_path}'\n")
361
362         logger.info("Stitching clips together...")
363         # Stitch them together without re-encoding (since they were encoded with
identical settings)
364         concat_cmd = [
365             "ffmpeg", "-y",
366             "-f", "concat",
367             "-safe", "0",
368             "-i", list_file_path,
369             "-c", "copy",
370             output_path
371         ]
372
373         try:
374             subprocess.run(concat_cmd, capture_output=True, check=True)
375
376             # Final verification
377             if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
378                 raise RuntimeError(
379                     f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
380                     f"but output file is missing or empty: {output_path}"
381                 )
382
383             output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
384             logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
385             return output_path
386         except subprocess.CalledProcessError as e:
387             stderr_msg = e.stderr.decode(errors='replace').strip()
388             raise RuntimeError(
389                 f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
390                 f"Command: {' '.join(concat_cmd)}\n"
391                 f"FFmpeg stderr: {stderr_msg[-500:]}"
392             ) from e
393

```

4. assistant (2026-06-18T07:52:24.306Z)

Now I understand the compiler. The candidate claims that padding is applied per-clip AFTER `\_merge\_overlapping`, so adjacent rallies closer than the combined padding will overlap. I'm the REACHABILITY lens ? I need to prove a real path supplies two distinct, non-abutting rallies that are closer than the combined padding. Let me examine how `compile\_highlights` is called and what segments are passed.

5. user (2026-06-18T07:52:25.701Z)

```

tests\test_api_endpoints.py:210:         def compile_highlights(self, raw, pairs, name):
tests\test_api_endpoints.py:238:         def compile_highlights(self, raw, pairs, name):
scripts\run_process_and_stitch.py:153:             out_path = compiler.compile_highlights(video_path,
time_pairs, output_filename)

```



```
docs\CODE_MAP.md:66: -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
docs\CODE_MAP.md:114:[Omitted long matching line]
docs\CODE_MAP.md:117: - `main()` sets `global_config.output.output_dir = args.output_dir`
(typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir,
output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so
`--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
docs\CODE_MAP.md:190:[Omitted long matching line]
docs\CODE_MAP.md:286:- **`output_dir`** a real field on `OutputConfig` (default `"output"`);
the CLI `--output-dir` overrides it on the typed model in `main()`, and `compile_highlights`
reads the same `global_config.output.output_dir` ? honored end-to-end.
docs\CODE_MAP.md:314:| Change stitching padding | `config.json
stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; wiring
`@backend/main.py` (`/api/compile` + CLI trim); mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
docs\CODE_MAP.md:316:| Filter highlights by shot count | `config.json stitching.min_shot_count`
? enforced on BOTH `@backend/main.py::compile_highlights` (`/api/compile`) and
`@scripts/run_process_and_stitch.py` (segments without `shot_count` metadata are exempt) |
tests\test_compiler.py:53:         comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
tests\test_compiler.py:77:         out_path = comp.compile_highlights(str(raw), [(0.0, 5.0),
(10.0, 15.0)], "../reel.mp4")
tests\test_compiler.py:90:         comp.compile_highlights(str(raw), [], "reel.mp4")
tests\test_compiler.py:96:         comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0,
5.0)], "reel.mp4")
tools\generate_highlights.py:194:         ).compile_highlights(stitch_src, pairs, reel_name)
backend\main.py:573:def compile_highlights(req: CompileRequest):
backend\main.py:623:         out_path = compiler.compile_highlights(video["filepath"],
time_pairs, out_name)
backend\main.py:758:         res_path = compiler.compile_highlights(args.video_path,
[(args.trim_start, args.trim_end)], out_name)
backend\pipeline\stitcher\compiler.py:190:     def compile_highlights(self, raw_video_path: str,
segments: List[Tuple[float, float]], output_filename: str) -> str:
tests\test_generate_highlights.py:20:         def compile_highlights(self, raw_video_path,
segments, output_filename):
tests\test_watermark.py:4:the configurable string), and the end-to-end wiring through
compile_highlights with
tests\test_watermark.py:168:         comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0,
15.0)], "reel.mp4")
tests\test_watermark.py:186:         comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
tests\test_watermark.py:198:         out_path = comp.compile_highlights(str(raw), [(0.0, 5.0)],
"reel.mp4")
tests\test_watermark.py:230:         comp.compile_highlights(str(raw), [(0.0, 5.0)],
"reel.mp4")
```

6. user (2026-06-18T07:52:25.702Z)

```
docs\CODE_MAP.md:66: -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
docs\CODE_MAP.md:190:[Omitted long matching line]
docs\CODE_MAP.md:314:| Change stitching padding | `config.json
stitching.{begin_padding,end_padding}`; impl
`@backend/pipeline/stitcher/compiler.py::VideoCompiler.compile_highlights`; wiring
`@backend/main.py` (`/api/compile` + CLI trim); mirror
`@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING` |
docs\CODE_MAP.md:370:| `test_compiler.py` | `pipeline/stitcher/compiler.VideoCompiler`
(`_parse_time` formats, re-encode->concat path, boundary validation; ffmpeg/ffprobe mocked) |
docs\EXPERIMENT_HARNESS.md:200: diff where they agree/diverge ? two reels via the existing
`VideoCompiler` is the GPU-side follow-on).
tools\generate_highlights.py:13: duplicate rallies. Belt-and-suspenders with VideoCompiler's
overlap-merge (PR #180).
tools\generate_highlights.py:41:from backend.pipeline.stitcher.compiler import VideoCompiler
tools\generate_highlights.py:189:         VideoCompiler(
backend\main.py:31:from backend.pipeline.stitcher.compiler import VideoCompiler
backend\main.py:620:     compiler = VideoCompiler(output_dir,
```

```

begin_padding=stitching.begin_padding,
backend\main.py:755:         compiler = VideoCompiler(out_dir,
begin_padding=stitching.begin_padding,
backend\pipeline\stitcher\compiler.py:16:class VideoCompiler:
backend\pipeline\stitcher\compiler.py:94:         s = VideoCompiler._parse_time(raw_s)
backend\pipeline\stitcher\compiler.py:95:         e = VideoCompiler._parse_time(raw_e)
backend\tools\rally_slicer.py:29:# Match VideoCompiler defaults so sliced clips frame a rally
the same way.
tests\test_compiler.py:1:"""VideoCompiler coverage (previously zero ? the user-facing highlight-
reel output path,
tests\test_compiler.py:9:from backend.pipeline.stitcher.compiler import VideoCompiler
tests\test_compiler.py:13:    f = VideoCompiler._parse_time
tests\test_compiler.py:22:    m = VideoCompiler._merge_overlapping
tests\test_compiler.py:35:    comp = VideoCompiler(str(tmp_path / "out"))
tests\test_compiler.py:59:    comp = VideoCompiler(str(tmp_path / "out"))
tests\test_compiler.py:86:    comp = VideoCompiler(str(tmp_path / "out"))
tests\test_compiler.py:94:    comp = VideoCompiler(str(tmp_path / "out"))
tests\test_api_endpoints.py:213:    mp.setattr(m, "VideoCompiler", _FakeCompiler)
tests\test_api_endpoints.py:242:    mp.setattr(m, "VideoCompiler", _FakeCompiler)
tests\test_api_endpoints.py:265:    VideoCompiler without the paddings, so reels always used the
constructor
tests\test_generate_highlights.py:13:    """A stand-in VideoCompiler that records its ctor
kwargs + compile args and writes the
tests\test_generate_highlights.py:116:    captured VideoCompiler ctor+compile args and the
compute_video_id mock."""
tests\test_generate_highlights.py:132:    patch.object(gh, "VideoCompiler",
_fake_compiler(captured)), \
tests\test_generate_highlights.py:142:    """REGRESSION: the runner must pass
stitching.watermark + begin/end_padding to VideoCompiler
tests\test_generate_highlights.py:143:    (it previously constructed VideoCompiler bare -> reels
shipped un-watermarked)."""
scripts\run_process_and_stitch.py:18:from backend.pipeline.stitcher.compiler import
VideoCompiler
scripts\run_process_and_stitch.py:150:    compiler = VideoCompiler(output_dir,
end_padding=end_padding, begin_padding=begin_padding,
tests\test_watermark.py:17:from backend.pipeline.stitcher.compiler import VideoCompiler
tests\test_watermark.py:44:    assert VideoCompiler(str(tmp_path / "a"),
watermark=None)._watermark_filter(str(tmp_path)) is None
tests\test_watermark.py:45:    off = VideoCompiler(str(tmp_path / "b"), watermark={"enabled":
False})
tests\test_watermark.py:51:    comp = VideoCompiler(str(tmp_path / "o"), watermark={"enabled":
True, "text": "custom"})
tests\test_watermark.py:64:    comp = VideoCompiler(str(tmp_path / "o"),
tests\test_watermark.py:71:    comp = VideoCompiler(str(tmp_path / "o1"),
tests\test_watermark.py:79:    comp = VideoCompiler(str(tmp_path / "o2"), watermark={"enabled":
True, "text": "X"})
tests\test_watermark.py:97:    comp = VideoCompiler(str(tmp_path / "o3"),
tests\test_watermark.py:111:    assert VideoCompiler._ff_quote(r"C:\fonts\Brand.ttf") ==
r"C:\:/fonts/Brand.ttf"
tests\test_watermark.py:113:    assert VideoCompiler._ff_quote("/usr/share/fonts/font.ttf") ==
"/usr/share/fonts/font.ttf"
tests\test_watermark.py:115:    assert VideoCompiler._ff_quote(r"D:\a\b\f.ttf") ==
r"D:\:/a\b/f.ttf"
tests\test_watermark.py:120:    comp = VideoCompiler(str(tmp_path / "o"),
tests\test_watermark.py:130:    comp = VideoCompiler(str(tmp_path / "o"),
tests\test_watermark.py:139:    comp = VideoCompiler(str(tmp_path / "o"),
tests\test_watermark.py:164:    comp = VideoCompiler(str(tmp_path / "out"),
tests\test_watermark.py:183:    comp = VideoCompiler(str(tmp_path / "out")) # no watermark
passed
tests\test_watermark.py:194:    comp = VideoCompiler(str(tmp_path / "out"),
tests\test_watermark.py:215:    comp = VideoCompiler(str(tmp_path / "out"),
watermark={"enabled": True, "text": "Y"})

```

The candidate's claim about the overlap is technically correct logic. But the REACHABILITY lens requires me to prove a real path supplies two genuinely distinct rallies closer than the combined padding. Let me look at how `time\_pairs` / segments are produced before reaching `compile\_highlights`. Let me look at the main entry points.

8. user (2026-06-18T07:52:30.542Z)

```
573     def compile_highlights(req: CompileRequest):
574         video = db_instance.get_video(req.video_id)
575         if not video:
576             raise HTTPException(status_code=404, detail="Indexed video record not found.")
577
578         # Retrieve matching play segments from db
579         all_segments = db_instance.query_play_segments(req.video_id, {})
580         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582         if not matched_segments:
583             raise HTTPException(status_code=400, detail="No matching play segments found to
584 stitch.")
585
586         # Reject path traversal / absolute output names (os.path.join would otherwise let an
587 # absolute or '..'-laden output_filename write anywhere on disk).
588         try:
589             out_name = safe_output_filename(req.output_filename)
590         except ValueError as e:
591             raise HTTPException(status_code=400, detail=str(e)) from e
592
593         # The configured shot-count floor (stitching.min_shot_count) applies on the API
594         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
595 shot_count
596         # metadata are exempt: the floor filters known counts only, so explicitly
597 # requested manual/legacy segments can't silently vanish under the default floor.
598 stitching = getattr(global_config, "stitching", None) or StitchingConfig()
599 kept = []
600 for s in matched_segments:
601     meta = s.get("metadata") or {}
602     sc = meta.get("shot_count") if isinstance(meta, dict) else None
603     try:
604         if sc is not None and int(sc) < stitching.min_shot_count:
605             continue
606     except (TypeError, ValueError):
607         pass # unparseable count -> treat as unknown, keep
608     kept.append(s)
609 if len(kept) < len(matched_segments):
610     logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
611 dropped "
612 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
613 segments below the floor.")
614 if not kept:
615     raise HTTPException(status_code=400,
616 detail=f"All {len(matched_segments)} matching segments fall
617 below "
618 f"stitching.min_shot_count={stitching.min_shot_count}.")
619
620 # Extract start/end time pairs
621 time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
622
623 # Run compiler with the configured stitching paddings + optional branding watermark
624 output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
625 or "output"
626 compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
627 end_padding=stitching.end_padding,
628 watermark=stitching.watermark)
629 try:
```

```

623         out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
624         return {"status": "success", "filepath": out_path}
625     except Exception as e:
626         raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
627
628     # --- CLI Debug Utility & Orchestration ---
629     def run_cli_diagnose_clip(video_path: str, timestamp: float):
630         """CLI debugging utility to examine segment properties around a timestamp."""
631         print("=" * 60)
632         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
633         print("=" * 60)
634
635         cfg = load_config() # legacy wrapper still works, returns dict for now
636
637         # 1. Validation check diagnostics
638         print("[DIAGNOSTIC] Running validation framework...")
639         results = registry.run_all(video_path, cfg)
640         for r in results:
641             status_symbol = "[PASS]" if r.passed else "[FAIL]"
642             print(f"  {status_symbol} {r.validator_name}: {r.message}")

```

9. assistant (2026-06-18T07:52:38.077Z)

The segments come from ``db_instance.query_play_segments`` (real detector output stored in the DB). These are genuine rally segments, and they can absolutely be closer than 1.0s apart in raw time ? rallies in badminton happen with short inter-rally gaps. The ``time_pairs`` are passed directly to ``compile_highlights`` with no inter-segment gap enforcement.

Now let me confirm the overlap logic precisely and look at the test file to see if this is already guarded/tested.

10. user (2026-06-18T07:52:38.811Z)

```

1     """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2     where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3     import os
4     import subprocess
5     from unittest.mock import patch, MagicMock
6
7     import pytest
8
9     from backend.pipeline.stitcher.compiler import VideoCompiler
10
11
12     def test_parse_time_formats():
13         f = VideoCompiler._parse_time
14         assert f(12.5) == 12.5
15         assert f("12.5") == 12.5
16         assert f("1:30") == 90.0
17         assert f("1:00:00") == 3600.0
18         assert f("garbage") == 0.0
19
20
21     def test_merge_overlapping_collapses_duplicates_and_overlaps():
22         m = VideoCompiler._merge_overlapping
23         # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
24         # to one each.
25         assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [(28.2, 34.5),
26         (40.5, 45.5)]
27         # Unsorted input is sorted; an overlapping range extends the previous.
28         assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [(0.0, 5.0), (100.0,
29         120.0)]
30         # Genuinely distinct (gapped) rallies are preserved, not merged.
31         assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]

```

```

29         # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
30         assert m(["0:10", "0:20"), (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]
31
32
33     def test_compile_dedups_before_stitching(tmp_path):
34         """A segment list with every rally twice must yield ONE clip per unique rally."""
35         comp = VideoCompiler(str(tmp_path / "out"))
36         raw = tmp_path / "raw.mp4"
37         raw.write_bytes(b"x")
38         trims = []
39
40         def fake_run(cmd, *a, **k):
41             if cmd[0] == "ffprobe":
42                 return MagicMock(stdout="100.0\n", returncode=0)
43             out = cmd[-1]
44             if isinstance(out, str) and out.endswith(".mp4"):
45                 open(out, "wb").write(b"clip")
46                 # count per-clip trims (the concat list file input is not an .mp4 output)
47                 if "-i" in cmd and cmd.index("-i") + 1 == str(raw):
48                     trims.append(cmd)
49             return MagicMock(returncode=0, stdout=b"", stderr=b"")
50
51         dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
52         with patch.object(subprocess, "run", side_effect=fake_run):
53             comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
54             # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
55             assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"
56
57
58     def test_compile_basenames_output_and_stitches(tmp_path):
59         comp = VideoCompiler(str(tmp_path / "out"))
60         raw = tmp_path / "raw.mp4"
61         raw.write_bytes(b"x")
62
63         calls = []
64
65         def fake_run(cmd, *a, **k):
66             calls.append(cmd)
67             # Create whatever output file ffmpeg/ffprobe was told to write so existence
68             checks pass.
69             if cmd[0] == "ffprobe":
70                 return MagicMock(stdout="100.0\n", returncode=0)
71             out = cmd[-1]
72             if isinstance(out, str) and out.endswith(("mp4",)):
73                 open(out, "wb").write(b"clip")
74             return MagicMock(returncode=0, stdout=b"", stderr=b"")
75
76         with patch.object(subprocess, "run", side_effect=fake_run):
77             # Traversal in the name must be stripped (defense-in-depth basename).
78             out_path = comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)],
79             "../reel.mp4")
80
81             assert os.path.basename(out_path) == "reel.mp4"
82             assert os.path.dirname(out_path).endswith("out")
83             # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
84             assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3
85
86
87     def test_compile_raises_on_no_segments(tmp_path):
88         comp = VideoCompiler(str(tmp_path / "out"))
89         raw = tmp_path / "raw.mp4"
90         raw.write_bytes(b"x")
91         with pytest.raises(ValueError):
92             comp.compile_highlights(str(raw), [], "reel.mp4")

```

```

92
93     def test_compile_raises_when_source_missing(tmp_path):
94         comp = VideoCompiler(str(tmp_path / "out"))
95         with pytest.raises(FileNotFoundError):
96             comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
97

```

11. assistant (2026-06-18T07:52:48.491Z)

The tests confirm:

- `\_merge\_overlapping` collapses on raw boundaries only (line 28: `(0.0, 5.0), (6.0, 9.0)` stays as two distinct segments ? a 1.0s raw gap).
- Padding is applied per-clip after the merge.

So with `begin_padding=0.5 + end_padding=0.5` (note: defaults are actually `end_padding=1.0`, `begin_padding=0.5` per the constructor at line 17), two rallies like `(0.0, 5.0)` and `(6.0, 9.0)` would be trimmed to `[.-5.5]` and `[5.5-..]` ? exactly abut. But with `end_padding=1.0` default, clip 1 ends at `5.0+1.0=6.0` and clip 2 starts at `6.0-0.5=5.5`, so frames 5.5-6.0 overlap. The overlap is real.

Now the REACHABILITY question: do genuinely distinct rallies closer than ``begin_padding + end_padding`` reach this code from a real production path? The DB stores detected segments. Let me verify segments aren't gap-enforced upstream, and check whether the rally detector merges close segments. Let me look at how play segments are produced/stored.

12. user (2026-06-18T07:52:48.682Z)

```

config.json:30:         "dead_time_merge_gap": 2.0,
README.md:175:         "dead_time_merge_gap": 2.0,
backend\main.py:562: def query_play_segments(req: QueryRequest):
backend\config\models.py:122:     dead_time_merge_gap: float = 2.0
eval_baselines\nightly_baseline.json:163:         "merge_gap": 2.0,
eval_baselines\nightly_baseline.json:328:         "merge_gap": 2.0,
eval_baselines\nightly_baseline.json:493:         "merge_gap": 2.0,
backend\infrastructure\database.py:311:     def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
backend\infrastructure\database.py:409:     def query_play_segments(self, video_id: str, filters:
Dict[str, Any]) -> List[Dict[str, Any]]:
backend\eval\calibrate_local.py:6:     WASB trajectory CSV ->
trajectory_to_action_windows(velocity, merge_gap, min_dur)
backend\eval\calibrate_local.py:34: # (velocity_thresh, merge_gap, min_window_duration,
min_crossings)
backend\eval\calibrate_local.py:48:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd,
backend\eval\calibrate_wasb.py:28: Config = Tuple[float, float, float] # (velocity_thresh,
merge_gap, min_window_duration)
backend\eval\calibrate_wasb.py:52:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd,
backend\eval\distill_local.py:43: # candidates (the classifier removes the junk).
(velocity_thresh, merge_gap, min_dur).
backend\eval\distill_local.py:59:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\eval\distill_local.py:81:         velocity_thresh=vt, merge_gap=mg,
min_window_duration=mwd)
backend\eval\distill_local.py:100:     return merge_overlaps(pos, gap_tol=1.0)
training\gen0\harness.py:46: CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur":
1.0}
backend\pipeline\audio\hit_detector.py:107:         min_gap_sec: float = 0.08, band:
Optional[Tuple[float, float]] = None) -> List[HitEvent]:
backend\pipeline\audio\hit_detector.py:109:     adaptive threshold (moving mean + k.std), keeping
the strongest within `min_gap_sec`.
backend\pipeline\audio\hit_detector.py:122:     min_gap = max(1, int(round(min_gap_sec /
hop_sec)))
backend\pipeline\audio\hit_detector.py:127:         lo, hi = max(0, i - min_gap), min(len(env), i

```

```

+ min_gap + 1)
docs\CODE_MAP.md:87: -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width,
chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic
WINDOWING BRAIN]
docs\CODE_MAP.md:129:[Omitted long matching line]
docs\CODE_MAP.md:176:[Omitted long matching line]
docs\CODE_MAP.md:178:[Omitted long matching line]
docs\CODE_MAP.md:179:[Omitted long matching line]
docs\CODE_MAP.md:181:[Omitted long matching line]
docs\CODE_MAP.md:237:`detection_params` keys by detector:
wasb={`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo={`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
docs\CODE_MAP.md:312:| Tune candidate windowing (velocity/merge/min-dur) |
`@backend\pipeline\detectors\tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`,`dead_time_merge_gap`,
`min_action_window_duration` |
backend\pipeline\detectors\tracknet_runner.py:262:         merge_gap: float = 2.0,
backend\pipeline\detectors\tracknet_runner.py:279:         ``merge_gap`` seconds of each other
are grouped into one window; short
backend\pipeline\detectors\tracknet_runner.py:334:         max_gap_frames = int(merge_gap * fps)
backend\pipeline\detectors\person_detector.py:141:
velocity_thresh: float, merge_gap: float,
backend\pipeline\detectors\person_detector.py:227:         max_gap_frames = int(merge_gap * fps)
docs\ANALYZERS_AND_RECONCILERS.md:367:| 3 | `over_merged` | a dead-time gap **> ** `merge_gap` of
shuttle-inactivity sits **inside** one segment | **split** at the gap |
backend\eval\export_wasb_segments.py:35:# The tuned config from the first per-video calibration
(velocity, merge_gap, min_dur);
backend\eval\export_wasb_segments.py:204:     ap.add_argument("--merge-gap", type=float,
default=TUNED[1])
backend\eval\export_wasb_segments.py:219:     cfg = (args.velocity_thresh, args.merge_gap,
args.min_duration)
backend\eval\experiment.py:103:         - ``threshold`` ? classifier probability cutoff.
``merge_gap`` ? post-filter
backend\eval\experiment.py:104:         overlap-merge gap (seconds), the same
`gemini_refine.merge_overlaps` default.
backend\eval\experiment.py:116:     merge_gap: float = 1.0
backend\eval\experiment.py:138:         "merge_gap": self.merge_gap,
backend\eval\experiment.py:258:     return merge_overlaps(kept, gap_tol=variant.merge_gap)
backend\eval\eval_partial_labels.py:132:# min_window_duration DOWN (recover short 3-5s rallies)
and merge_gap UP (heal
backend\eval\eval_partial_labels.py:146:     """Grid-search (velocity, merge_gap, min_dur) x gate
vs human labels, in-window.
backend\eval\eval_partial_labels.py:186:     print(f"\nbest: velocity={b['cfg'][0]}
merge_gap={b['cfg'][1]} min_dur={b['cfg'][2]} "
backend\pipeline\segmenters\motion.py:44:         dead_time_merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
backend\pipeline\segmenters\motion.py:141:         # Post-Processing: 1. Merge segments separated
by less than dead_time_merge_gap
backend\pipeline\segmenters\motion.py:146:         if next_start - curr_end <
dead_time_merge_gap:
backend\pipeline\segmenters\trajectory_hybrid.py:161:         merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
backend\pipeline\segmenters\trajectory_hybrid.py:180:         "merge_gap": merge_gap,
backend\pipeline\segmenters\trajectory_hybrid.py:208:
velocity_thresh=velocity_thresh, merge_gap=merge_gap,
backend\pipeline\segmenters\yolo_hybrid.py:100:     merge_gap =
idx_cfg.get("dead_time_merge_gap", 2.0)
backend\pipeline\segmenters\yolo_hybrid.py:112:         "merge_gap": merge_gap,
backend\pipeline\segmenters\yolo_hybrid.py:171:         chunk_path, chunk_start,
velocity_thresh, merge_gap,
backend\pipeline\segmenters\yolo_hybrid.py:196:         chunk_path, chunk_start,
velocity_thresh, merge_gap,
backend\pipeline\segmenters\wasb_hybrid.py:8:candidate windows); the windowing thresholds
(velocity_thresh / merge_gap /

```

```

backend\eval\golden_fixtures.py:82:#: The pinned SERVED windowing the synthetic fixtures freeze
at: (velocity_thresh, merge_gap,
backend\eval\golden_fixtures.py:263: # low-velocity gap (> merge_gap=2.0s) ? proves dead-
time separates windows.
tests\test_compiler.py:27: # Genuinely distinct (gapped) rallies are preserved, not merged.
backend\eval\gemini_refine.py:39:def merge_overlaps(intervals: List[Interval], gap_tol: float =
0.0) -> List[Interval]:
backend\eval\gemini_refine.py:42: gap_tol > 0 merges two intervals separated by <= gap_tol
seconds ? used to heal brief
backend\eval\gemini_refine.py:48: if merged and s <= merged[-1][1] + gap_tol:
backend\eval\gemini_refine.py:95: # (merge_gap bridged the between-point gaps), and
collapsing them to one envelope swept
backend\eval\gemini_refine.py:97: # by the gap-tolerant global merge in run().
backend\eval\gemini_refine.py:133: refined = [(s, e) for s, e in
merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >= min_dur]
backend\eval\gemini_refine.py:188: rallies = [(s, e) for s, e in merge_overlaps([iv for c in
chunks for iv in c], gap_tol=1.0) if e - s >= min_dur]
backend\eval\rally_seg_eval.py:231: f"(vt={p['velocity_thresh']},
merge_gap={p['merge_gap']}, "
backend\eval\rally_seq_proto.py:19: -> merge tiny gaps / drop short -> score vs GT at
IoU>=0.5 (same metric as the heuristic).
backend\eval\rally_seq_proto.py:78: merge_gap: float = 1.0, min_dur: float
= 1.0, smooth: int = 7) -> List[Interval]:
backend\eval\rally_seq_proto.py:94: if merged and s - merged[-1][1] <= merge_gap:
backend\eval\segmentation_metrics.py:4:chopped into three predicted pieces, or two rallies
merged across a dead-time gap, can
scripts\nightly_regression.py:119: All three are anchored on the SAME ``windowing.SERVED``
base knobs (velocity_thresh / merge_gap /
backend\eval\served_gate_a.py:9:(``velocity_threshold=0.02, dead_time_merge_gap=2.0,
min_action_window_duration=2.0``), the
backend\eval\served_gate_a.py:54: "dead_time_merge_gap": PROD_MERGE_GAP,
backend\eval\served_gate_a.py:71: velocity_thresh=PROD_VELOCITY_THRESH,
merge_gap=PROD_MERGE_GAP,
backend\eval\serve_contrast.py:46: min_crossings: int = MIN_CROSSINGS,
merge_gap: float = 1.0
backend\eval\serve_contrast.py:54: ``merge_gap`` is the post-keep overlap-merge gap (the
harness's ``merge_overlaps`` default),
backend\eval\serve_contrast.py:64: control = merge_overlaps(intervals, gap_tol=merge_gap)
backend\eval\serve_contrast.py:66: gate_a = merge_overlaps(kept, gap_tol=merge_gap)
tests\test_experiment.py:105: # merge_gap=1.0 stitches the 0.5s gap between the first two;
the 28s gap stays split.
tests\test_gemini_refine.py:11:def
test_merge_overlaps_gap_tol_stitches_small_gaps_keeps_real_ones():
tests\test_gemini_refine.py:13: assert gr.merge_overlaps([(1, 3), (3.5, 5), (10, 12)],
gap_tol=1.0) == [(1, 5), (10, 12)]
tests\test_gemini_refine.py:14: # without tolerance the 0.5s gap is preserved (no over-merge)
tests\test_golden_fixtures.py:104: # interior hole would not move start/end, since merge_gap
coalesces across it and the frame
docs\EXPERIMENT_HARNESS.md:100:- `backend/eval/calibrate_local.py` ? `run(specs, iou=0.5,
metric="f1")` (:77); `LocalConfig = (velocity_thresh, merge_gap, min_window_duration,
min_crossings)`; grid + LOVO.
docs\EXPERIMENT_HARNESS.md:106:- `backend/config/models.py::IndexingConfig` ?
`default_segmenter` (:65, default ``yolo_hybrid``), `detector_model` (:72),
`detector_score_threshold` (:73), `yolo_velocity_threshold` (:74), `yolo_frame_skip` (:75),
`min_segment_duration` (:68), `dead_time_merge_gap` (:69), `motion_threshold` (:67). The
`min_crossings` knob now lives on `FusionConfig` (default `0` = OFF; also `min_players`, both
served Gate A/B thresholds ? see fusion P2).
docs\EVALUATION.md:104: `dead_time_merge_gap`, `min_action_window_duration`) or the AI prompt ?
no new
backend\eval>windowing.py:5:on a *recall-oriented PROPOSAL* windowing (`vt=0.005,
merge_gap=1.0, min_window=0.5`):
backend\eval>windowing.py:7:(`TrajectoryHybridSegmenter.process_video`) windows COARSELY
(`vt=0.02, merge_gap=2.0,
backend\eval>windowing.py:49: """A named (velocity_thresh, merge_gap, min_window_duration)
windowing operating point.

```



```

backend\eval\windowing.py:52: "in flight"; ``merge_gap`` ? seconds within which active frames
coalesce into one window;
backend\eval\windowing.py:59: merge_gap: float
backend\eval\windowing.py:62: #: inactivity gap ? the bounded-windowing over-merge fix (w1,
RALLY_QUALITY_RESEARCH.md §6).
backend\eval\windowing.py:81: """"(velocity_thresh, merge_gap, min_window_duration)`` ?
the legacy tuple shape the
backend\eval\windowing.py:84: return (self.velocity_thresh, self.merge_gap,
self.min_window_duration)
backend\eval\windowing.py:90: "merge_gap": self.merge_gap,
backend\eval\windowing.py:107: - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
backend\eval\windowing.py:118: merge_gap=float(idx_cfg.dead_time_merge_gap),
backend\eval\windowing.py:139: name="proposal", velocity_thresh=0.005, merge_gap=1.0,
min_window_duration=0.5
backend\eval\windowing.py:160: merge_gap=float(idx_cfg.get("dead_time_merge_gap",
SERVED.merge_gap)),
backend\eval\windowing.py:198: velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
docs\archives\decisions\DECISIONS.md:208: as the merge gap grows ? windows overshoot rather
than align).
docs\HOW_RALLY_DETECTION_WORKS.md:36:- ``merge_gap`` ? active frames within this many seconds
get joined into one rally (bridges
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:51:Best by F1 (fit-on-self):
``velocity 0.03 / merge_gap 1.0 / min_dur 2.0 / gate OFF ?
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:59:| ``merge_gap`` | Best stays
at 1.0; raising it to heal fragmentation merged adjacent distinct rallies and hurt precision
(see failure mode below). |
docs\archives\quality-iterations\2026-06-human-gt-tuning\README.md:68: invisibility gap exceeds
any sane ``merge_gap``, so raising it doesn't help (it just merges
docs\QUALITY_ITERATIONS.md:200:### REAL-FOOTAGE VALIDATION + first ground-truth at-par ? over-
merge fix confirmed; the gap is the rally-END *(2026-06-16)*
docs\QUALITY_ITERATIONS.md:367:"bridge the local?Gemini gap"). **Why:** plain temporal-IoU F1 is
nearly blind to the *over-merge* the local
docs\QUALITY_ITERATIONS.md:400:(``velocity_thresh=0.02, merge_gap=2.0, min_window=2.0``) ? no WASB
re-run (the runner has no cache-hit early-return), no GPU, no
docs\QUALITY_ITERATIONS.md:503:permissive PROPOSAL windowing`` (``velocity_threshold=0.005,
merge_gap=1.0, min_window=0.5``), which emits many
docs\QUALITY_ITERATIONS.md:505:(``velocity_threshold=0.02, merge_gap=2.0, min_window=2.0``)
already merges flight bursts into a few long windows
docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md:16: proposal windowing
``velocity_thresh=0.005, merge_gap=1.0, min_dur=0.5``) ? per-window shuttle features
docs\TRACKNET_WSL_SETUP.md:81: ``dead_time_merge_gap``. Raw candidates are persisted for fine-
tuning (same table
tests\test_nightly_regression.py:28:_PRESET = {"name": "x", "velocity_thresh": 0.02,
"merge_gap": 2.0, "min_window_duration": 2.0,
tests\test_nightly_regression.py:307: assert b.merge_gap == cfgs["default"].preset.merge_gap
docs\README.md:2:[Omitted long matching line]
docs\REAL_FOOTAGE_VALIDATION_REPORT.md:11:[Omitted long matching line]
docs\REAL_FOOTAGE_VALIDATION_REPORT.md:49:[Omitted long matching line]
docs\REAL_FOOTAGE_VALIDATION_REPORT.md:82:| **R3** | **Re-scope the cap to true rally length** ?
test ``max_window`` in the **6?8s** band (+ separate end-trim) instead of 12s; the hard-cap lift
already signals 12s is coarse. Ablate cap-only vs hard-cap-chop on the GT clip, holding
velocity_thresh/merge_gap fixed. | S?M | Less END over-extension and less over-counting on
short-rally venues | Medium (12s coarseness measured; 6?8s untested) | R2 (need the metric to
score it) |
docs\REAL_FOOTAGE_VALIDATION_REPORT.md:113:[Omitted long matching line]
docs\RALLY_DETECTION_QUALITY_REPORT.md:116: inactivity gap (the See-et-al. sustained-inactivity
boundary). Fixes the over-merge.
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:30:slice and the full match**
(merge_gap 1.0, min_dur 2.0; velocity irrelevant) ? tuning is stable. Per-rally:
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:227:- **Tuning is real but
footage-bounded** ? ``merge_gap 1.0 / min_dur 2.0`` transfers; the gate trades recall?precision;
docs\RALLY_DETECTION_GAP_CLOSURE.md:12:[Omitted long matching line]
docs\RALLY_QUALITY_RESEARCH.md:84:- Active frames within ``dead_time_merge_gap`` (2.0 s) are
**merged** into one window.
docs\RALLY_QUALITY_RESEARCH.md:89:slow serve prep) ? those frames read as *active*. The 2 s

```

```

merge then bridges the brief gaps, and
docs\RALLY_QUALITY_RESEARCH.md:127: learned-classifier + threshold/merge_gap/calibrate),
`compare()` (LOVO-honest A/B with an
docs\RALLY_QUALITY_RESEARCH.md:246:our heuristic has none (it merges on a velocity+gap rule).
The transferable *idea* is "predict a
docs\RALLY_QUALITY_RESEARCH.md:462: it scored 0.157 on multi-court). Tune `velocity_threshold`
+ `merge_gap` so local segment counts
docs\RALLY_QUALITY_RESEARCH.md:536:| **W1.5** | Hard-cap fallback (`window_hard_cap`) ? chop
continuously-active windows at the cap when no inactivity gap exists | 1 | W1 | golden ?F1 ?0
(or neutral) **** fixes a real-footage over-merge it can't otherwise cap | ? **golden ?F1
+0.006 (n.s.)**; real `mahadevpura-1` 174s blob ? 24x?9s; retained **default-OFF** (real-footage
+ true-cap rationale, not a golden lift) |
tests\test_person_detector.py:45: "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05,
merge_gap=2.0,
tests\test_rally_seq_proto.py:82: segs = rp.probs_to_segments(probs, times, threshold=0.5,
merge_gap=1.0, min_dur=1.0, smooth=1)
tests\test_rally_seg_eval.py:11:_PRESET = WindowingPreset(name="t", velocity_thresh=0.02,
merge_gap=2.0, min_window_duration=0.5)
tests\test_served_gate_a_regression.py:18:(velocity_threshold 0.02, merge_gap 2.0, min_window
2.0) ? few long windows, over-seg ratio
tests\test_served_gate_a_regression.py:144: merge_gap=_PROPOSAL[1],
min_window_duration=_PROPOSAL[2])
tests\test_segmentation_metrics.py:33: # Two rallies merged across the dead-time gap ? low
IoU with either, both missed.
tests\test_reporting_harvest.py:22: def query_play_segments(self, video_id, filters):
tests\test_tracknet_runner.py:80: merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
tests\test_tracknet_runner.py:95: merge_gap=2.0, min_window_duration=0.5,
tests\test_tracknet_runner.py:108: velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3,
tests\test_tracknet_runner.py:123:# Two rallies merged across a sub-merge_gap dead time ? ONE
over-long window without the cap.
tests\test_tracknet_runner.py:131: merge_gap=2.0, min_window_duration=0.3)
tests\test_tracknet_runner.py:137: # With a 4s cap the 8s merged window is split at its 1.4s
internal dead-time gap ? 2 windows.
tests\test_tracknet_runner.py:140: merge_gap=2.0, min_window_duration=0.3,
max_window_duration=4.0, min_split_gap=0.5)
tests\test_tracknet_runner.py:151: merge_gap=2.0, min_window_duration=0.3,
max_window_duration=4.0, min_split_gap=0.5)
tests\test_tracknet_runner.py:172: merge_gap=2.0, min_window_duration=0.3,
max_window_duration=4.0, min_split_gap=0.5,
tests\test_tracknet_runner.py:212: pts, fps=30.0, frame_width=1280.0,
velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3)
tests\test_tracknet_runner.py:214: pts, fps=30.0, frame_width=1280.0,
velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
tests\test_tracknet_runner.py:242: pts, fps=30.0, frame_width=1280.0,
velocity_thresh=0.02, merge_gap=2.0,
tests\test_tracknet_runner.py:245: pts, fps=30.0, frame_width=1280.0,
velocity_thresh=0.02, merge_gap=2.0,
tests\test_tracknet_runner.py:257: _moving(0, 301), fps=30.0, frame_width=1280.0,
velocity_thresh=0.02, merge_gap=2.0,
tests\test_windowing.py:29: # 8-30: occluded (a long gap so the burst and the rally never
merge into one window).
tests\test_windowing.py:46: assert W.SERVED.merge_gap == idx.dead_time_merge_gap
tests\test_windowing.py:87: "dead_time_merge_gap": 1.5,
tests\test_windowing.py:108: assert d == {"name": "served", "velocity_thresh": 0.02,
"merge_gap": 2.0,
tests\test_yolo_hybrid.py:78: "dead_time_merge_gap": 2.0,
tests\test_yolo_hybrid.py:146: "dead_time_merge_gap": 2.0,
tests\test_yolo_hybrid.py:172: "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05,
merge_gap=2.0,
tests\test_yolo_hybrid.py:215: "dead_time_merge_gap": 2.0,
tests\test_yolo_hybrid.py:265: "dead_time_merge_gap": 2.0,

```

The key fact: the segmenters merge active windows within `dead\_time\_merge\_gap` (default 2.0s) at detection time. So at the detector level, two distinct rallies separated by less than 2.0s of dead-time get merged into one window. This means the segments stored in the DB are already separated by >2.0s of dead-time gap in the typical case.

But this is the detector windowing. The crucial question is whether segments separated by a raw gap < 1.5s (begin_padding 0.5 + end_padding 1.0 = 1.5s default) can actually reach `compile\_highlights`. Let me check the segmenter post-processing to understand exactly the minimum gap between persisted segments.

14. user (2026-06-18T07:52:55.238Z)

```
135         raw_segments.append((segment_start, t))
136
137     # Handle video ending while in active segment
138     if in_segment:
139         raw_segments.append((segment_start, timestamps[-1]))
140
141     # Post-Processing: 1. Merge segments separated by less than dead_time_merge_gap
142     merged_segments: List[Tuple[float, float]] = []
143     if raw_segments:
144         curr_start, curr_end = raw_segments[0]
145         for next_start, next_end in raw_segments[1:]:
146             if next_start - curr_end < dead_time_merge_gap:
147                 # Merge
148                 curr_end = next_end
149             else:
150                 merged_segments.append((curr_start, curr_end))
151                 curr_start, curr_end = next_start, next_end
152         merged_segments.append((curr_start, curr_end))
153
154     # Post-Processing: 2. Filter segments shorter than min_segment_duration
155     final_segments: List[Tuple[float, float]] = [
156         r for r in merged_segments if (r[1] - r[0]) >= min_segment_duration
157     ]
158
159     # Populate SQLite DB with segments and metadata events
160     segments_data = []
161     for start, end in final_segments:
162         # Assign Server and Receiver randomly from pool
163         server, receiver = random.sample(player_pool, 2)
164
```

15. user (2026-06-18T07:52:56.311Z)

```
320         prev = None # break the trajectory; absent shuttle = not in flight
321         continue
322     if prev is not None:
323         dframes = p.frame - prev.frame
324         if dframes > 0:
325             dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326             velocity = (dist / frame_width) * (fps / dframes)
327             if velocity > velocity_thresh:
328                 active.append((p.frame, velocity))
329         prev = p
330
331     if not active:
332         return []
333
334     max_gap_frames = int(merge_gap * fps)
335
336     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337         vels = [v for _, v in group]
338         return {
339             "start": group[0][0] / fps + chunk_start,
```

```

340         "end": group[-1][0] / fps + chunk_start,
341         "active_frame_count": len(group),
342         "peak_velocity": max(vels),
343         "mean_velocity": sum(vels) / len(vels),
344         "track_id_count": 1, # single shuttle; kept for window-shape parity
345         "chunk_index": chunk_index,
346     }
347
348     groups: List[List[Tuple[int, float]]] = []
349     group = [active[0]]
350     for af in active[1:]:
351         if af[0] - group[-1][0] <= max_gap_frames:
352             group.append(af)
353         else:
354             groups.append(group)
355             group = [af]
356     groups.append(group)
357
358     if max_window_duration > 0:
359         groups = TrackNetRunner._split_overlong_groups(
360             groups, int(max_window_duration * fps), int(min_split_gap * fps),
361             hard_cap=window_hard_cap)
362
363     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368     if window_inactivity_end > 0:
369         tw = int(window_inactivity_end * fps)
370         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371             inactivity_rally_thresh) for g
in groups]
372
373     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374     # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375     # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376     # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377     forced_flags = [False] * len(groups)
378     if window_blob_fallback and max_window_duration > 0:
379         groups, forced_flags = TrackNetRunner._blob_fallback_chop(

```

16. assistant (2026-06-18T07:53:00.906Z)

This is a critical finding for reachability. The `max\_window\_duration` cap (default-OFF generally, but `\_split\_overlong\_groups` and R5 `\_blob\_fallback\_chop`) can split one continuously-active window into multiple sub-windows. Let me check the split logic ? specifically what gap it leaves between the split pieces. If a continuously-active blob is force-chopped at the midpoint, the pieces are ADJACENT (zero gap or a tiny `min\_split\_gap`).

Let me read `\_split\_overlong\_groups` and `\_blob\_fallback\_chop`.

17. user (2026-06-18T07:53:01.824Z)

```

backend\pipeline\detectors\tracknet_runner.py:359:         groups =
TrackNetRunner._split_overlong_groups(

```

```

backend\pipeline\detectors\tracknet_runner.py:379:         groups, forced_flags =
TrackNetRunner._blob_fallback_chop(
backend\pipeline\detectors\tracknet_runner.py:391:     def _trim_inactive_tail(group:
List[Tuple[int, float]], trail_frames: int,
backend\pipeline\detectors\tracknet_runner.py:420:     def _split_overlong_groups(groups:
List[List[Tuple[int, float]]], max_win_frames: int,
backend\pipeline\detectors\tracknet_runner.py:461:     def _blob_fallback_chop(groups:
List[List[Tuple[int, float]]], max_win_frames: int,
docs\QUALITY_ITERATIONS.md:77:two-sided motion ? not a bigger cap). `_blob_fallback_chop` in
`tracknet_runner`, wired through
docs\QUALITY_ITERATIONS.md:273:`trajectory_to_action_windows` ?
`_split_overlong_groups(hard_cap=)`, `WindowingPreset.hard_cap`,
tests\test_tracknet_runner.py:156:def test_split_overlong_groups_pure():
tests\test_tracknet_runner.py:159:     out = TrackNetRunner._split_overlong_groups([grp],
max_win_frames=30, min_split_frames=10)
tests\test_tracknet_runner.py:163:     assert TrackNetRunner._split_overlong_groups([solid], 30,
10) == [solid]
tests\test_tracknet_runner.py:179:def test_split_overlong_groups_hard_cap_pure():
tests\test_tracknet_runner.py:182:     assert TrackNetRunner._split_overlong_groups([solid], 30,
10) == [solid]
tests\test_tracknet_runner.py:183:     out = TrackNetRunner._split_overlong_groups([solid], 30,
10, hard_cap=True)
tests\test_tracknet_runner.py:222:def test_blob_fallback_chop_pure():
tests\test_tracknet_runner.py:224:     out, flags = TrackNetRunner._blob_fallback_chop([blob],
max_win_frames=30, blob_factor=4.0)
tests\test_tracknet_runner.py:234:     out, flags = TrackNetRunner._blob_fallback_chop([normal],
max_win_frames=30, blob_factor=4.0)

```

18. user (2026-06-18T07:53:05.115Z)

```

420         def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                                     min_split_frames: int,
422                                     hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
423             """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424             internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425             boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426             invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427             rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
428             temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
429             bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430             if max_win_frames <= 0:
431                 return groups
432             out: List[List[Tuple[int, float]]] = []
433             work = list(groups)
434             while work:
435                 g = work.pop()
436                 if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                     out.append(g)
438                     continue
439                 best_i, best_gap = -1, -1
440                 for i in range(1, len(g)):
441                     gap = g[i][0] - g[i - 1][0]
442                     if gap > best_gap:
443                         best_gap, best_i = gap, i
444                 if best_i < 0 or best_gap < min_split_frames:
445                     # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).

```

```

446         # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447         if hard_cap:
448             mid = (g[0][0] + g[-1][0]) / 2.0
449             split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450             work.append(g[:split_i])
451             work.append(g[split_i:])
452         else:
453             out.append(g)
454             continue
455         work.append(g[:best_i])
456         work.append(g[best_i:])
457     out.sort(key=lambda grp: grp[0][0])
458     return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
466         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
467         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
469         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
472         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474         that piece came from a forced chop. Iterative worklist to bound stack depth."""
475         if max_win_frames <= 0:
476             return groups, [False] * len(groups)
477         blob_frames = max_win_frames * max(1.0, blob_factor)
478         out: List[List[Tuple[int, float]]] = []
479         flags: List[bool] = []
480         forced = 0
481         for g in groups:
482             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483                 out.append(g)
484                 flags.append(False)
485                 continue
486             pieces: List[List[Tuple[int, float]]] = []
487             work = [g]
488             while work:
489                 h = work.pop()
490                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491                     pieces.append(h)
492                     continue
493                 mid = (h[0][0] + h[-1][0]) / 2.0
494                 split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
495                 work.append(h[:split_i])
496                 work.append(h[split_i:])
497             pieces.sort(key=lambda grp: grp[0][0])
498             out.extend(pieces)
499             flags.extend([True] * len(pieces))

```

```

500         forced += len(pieces) - 1
501     if forced:
502         logger.warning(
503             "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505             "motion), not a bigger cap", forced)
506     return out, flags
507

```

19. assistant (2026-06-18T07:53:18.759Z)

This is the smoking gun for reachability. When `hard\_cap` or `\_blob\_fallback\_chop` splits a continuously-active group, the split is done at a frame index `split\_i`: `work.append(h[:split\_i])` and `work.append(h[split\_i:])`.

The first piece ends at frame `h[split\_i - 1][0]` and the second piece starts at frame `h[split\_i][0]`. These are **consecutive active frames** ? adjacent, with essentially zero gap between them (the group had no internal gap ? the split floor, which is why it was hard-chopped). So the two resulting windows are temporally adjacent: piece1 end ? piece2 start, separated by at most one inter-frame interval (1/fps ? 0.033s).

These then become two distinct DB segments (e.g., end=15.0, start=15.033). In `compile\_highlights`, `\_merge\_overlapping` keeps them as two distinct ranges (start 15.033 > previous end 15.0). Then padding makes clip1 end at 15.0+1.0=16.0 and clip2 start at 15.033-0.5=14.533 ? frames 14.533s to 16.0s overlap heavily and play twice.

However ? I need to check: are these split windows persisted as separate DB segments, and is hard_cap / blob_fallback actually reachable in production (default-ON)? The candidate's scenario doesn't depend on these (it's about generic raw rally proximity), but these features make even strongly-adjacent segments reachable. Let me verify the W1 gap-split case which is closer to default behavior. After W1 split at the largest internal gap (? `min\_split\_frames`), the gap between pieces is `best\_gap` frames. `min\_split\_gap` default ? let me check the production wiring of these knobs.

20. user (2026-06-18T07:53:18.899Z)

```

255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,
259         frame_width: float,
260         chunk_start: float = 0.0,
261         velocity_thresh: float = 0.02,
262         merge_gap: float = 2.0,
263         min_window_duration: float = 2.0,
264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267         window_hard_cap: bool = False,
268         window_inactivity_end: float = 0.0,
269         inactivity_min_density: float = 0.34,
270         inactivity_rally_thresh: float = 0.08,
271         window_blob_fallback: bool = False,
272         window_blob_factor: float = 4.0,
273     ) -> List[Dict[str, Any]]:
274         """Convert a shuttle trajectory into candidate rally windows.
275
276         A frame is "active" when the shuttle is visible AND its inter-frame
277         displacement (normalised by frame width, per second) exceeds
278         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279         ``merge_gap`` seconds of each other are grouped into one window; short
280         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281         shape feeding the AI-handoff phase is identical.

```

```

282
283     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
against
284     the *over-merge* failure where "shuttle moving" is conflated with "rally in
progress"
285     and a single window swallows several rallies + the dead time between them. When
286     window longer than this is recursively SPLIT at its largest internal inactivity
287     longest stretch with no in-flight shuttle ? the most likely rally boundary),
provided
288     that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
fix from
289     ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
recall cannot
290     drop, and it is config-gated default-OFF until measured on the golden set.
291
292     ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
293     over-long window has NO internal inactivity gap to split on (a continuously-
active
294     trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
295     blob where the gap-splitter alone left a single 174 s window), it is recursively
296     hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
297     (never merges), so recall cannot drop; gated default-OFF until measured.
298
299     ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
300     ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
301     over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
302     force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` x
303     ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
304     their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
305     alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
306     logged, and
307     each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
308     rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
309     can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
310     """
311     if fps <= 0:
312         fps = 30.0
313     if frame_width <= 0:
314         frame_width = 1280.0
315
316     # Compute per-frame normalised velocity from the last *visible* point.
317     active = [] # (frame_idx, velocity)
318     prev: Optional[TrajectoryPoint] = None
319     for p in points:
320         if not p.visible:

```

21. assistant (2026-06-18T07:53:29.503Z)

Now I understand the full picture. The W1 split (default-OFF), hard_cap (default-OFF), and blob_fallback (default-OFF) can all produce adjacent windows. But these are default-OFF. The candidate's claim, however, does NOT require any of these features ? it claims that two

genuinely distinct rallies with a raw gap < combined padding reach this code through the standard path.

The standard (default-ON) path is: detector ? windows merged within `dead\_time\_merge\_gap` (2.0s) ? persisted to DB ? `compile\_highlights`. With `dead\_time\_merge\_gap=2.0`, any two persisted segments are separated by **at least 2.0s** of raw gap (the merge step combines anything closer). With default padding begin=0.5 + end=1.0 = 1.5s combined, and a minimum 2.0s gap, the padded ranges do NOT overlap: clip1 ends at end+1.0, clip2 starts at start-0.5, and since start - end ? 2.0, padded gap = 2.0 - 1.5 = 0.5s > 0. ****No overlap on the default detector path.****

So the candidate's described path via the standard detector windowing is NOT reachable with defaults ? the dead-time merge guarantees a ?2.0s gap which exceeds the 1.5s combined padding.

Now let me check whether segments from OTHER sources can reach `compile\_highlights` with sub-1.5s gaps. The `api/compile` path pulls from the DB. Are there paths where segments come from somewhere else, or where `dead\_time\_merge\_gap` < combined padding, or where manual/legacy segments bypass the merge? Let me check the segment-correction / manual-add path and DB.

22. user (2026-06-18T07:53:30.957Z)

```
311         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
312             try:
313                 with self._connect() as conn:
314                     cursor = conn.cursor()
315                     cursor.execute(
316                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
317                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
318                     )
319                     conn.commit()
320                     return cursor.lastrowid
321             except Exception as e:
322                 logger.error(f"Failed to add play segment: {e}")
323                 return None
324
325         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
326             try:
327                 with self._connect() as conn:
328                     conn.cursor().execute(
329                         "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
330                         (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
331                     )
332                     conn.commit()
333                     return True
334             except Exception as e:
335                 logger.error(f"Failed to add event: {e}")
336                 return False
337
338         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
339                                     chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
340                                     stats: Optional[Dict[str, Any]] = None,
341                                     detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
342             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
343             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
344             try:
```

```

345         with self._connect() as conn:
346             cursor = conn.cursor()
347             cursor.execute(
348                 """INSERT INTO candidate_segments
349                     (video_id, detector, chunk_index, start_time, end_time, duration,
350                     confidence, stats, detection_params)
351                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
352                 (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
353                 confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
354             )
355             conn.commit()
356             return cursor.lastrowid
357     except Exception as e:
358         logger.error(f"Failed to add candidate segment: {e}")
359         return None
360
361     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
362         """Record that a candidate detection was confirmed as a given play_segment."""
363         try:
364             with self._connect() as conn:
365                 conn.cursor().execute(
366                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
? ",
367                     (segment_id, candidate_id)
368                 )
369                 conn.commit()
370             return True

```

23. assistant (2026-06-18T07:53:34.257Z)

`add\_play\_segment` is a raw INSERT with no gap enforcement. The crucial reachability question is whether any production path persists segments closer than the combined padding. Let me check who calls `add\_play\_segment` and whether there's a Gemini-refine path that produces segments separated by less than 1.5s. Let me look at the AI refinement path and where segments get added.

24. user (2026-06-18T07:53:34.936Z)

```

docs\CODE_MAP.md:74: -> P4 db.add_play_segment + db.link_candidate_to_segment
docs\CODE_MAP.md:76: Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate
(clamp/dedup) -> add_play_segment.
docs\CODE_MAP.md:216: [Omitted long matching line]
docs\CODE_MAP.md:239: $ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
tests\test_async_jobs.py:64:         sid = self.db.add_play_segment(video_id, 0.0, 5.0)
tests\test_async_jobs.py:73:         self.db.add_play_segment(video_id, 9.0, 10.0) # half-
written row pre-crash
tests\test_async_jobs.py:254:     db.add_play_segment(vid, 1.0, 2.0) # prior good result
tests\test_async_jobs.py:271:     db.add_play_segment(vid, 1.0, 2.0) # prior good result
tests\test_api_endpoints.py:56:         sid = self.db.add_play_segment(video_id, 0.0, 5.0)
tests\test_api_endpoints.py:86:     db.add_play_segment(vid, 1.0, 2.0) # prior good result
tests\test_api_endpoints.py:186:     db.add_play_segment("vid", 1.0, 4.0)
tests\test_api_endpoints.py:195:     sid = db.add_play_segment("vid", 1.0, 4.0)
tests\test_api_endpoints.py:204:     sid = db.add_play_segment("vid", 1.0, 4.0)
tests\test_api_endpoints.py:228:     s_low = db.add_play_segment("vid", 1.0, 4.0, None, None,
{"shot_count": 5})
tests\test_api_endpoints.py:229:     s_high = db.add_play_segment("vid", 10.0, 14.0, None, None,
{"shot_count": 12})
tests\test_api_endpoints.py:230:     s_unknown = db.add_play_segment("vid", 20.0, 24.0) # no
shot_count metadata
tests\test_api_endpoints.py:255:     sid = db.add_play_segment("vid", 1.0, 4.0, None, None,
{"shot_count": 3})
tests\test_api_endpoints.py:274:     sid = db.add_play_segment("vid", 10.0, 14.0)
tests\test_automated_provider.py:85:     d.add_play_segment("vid1", 0.0, 10.0)
tests\test_automated_provider.py:86:     d.add_play_segment("vid1", 20.0, 30.0)

```

```

backend\infrastructure\database.py:311:     def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
tests\test_database.py:36:     sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA",
"PlayerB", {"foo": "bar"})
tests\test_database.py:147:     sid = db.add_play_segment("vid1", 1.0, 4.0)
tests\test_eval_harness.py:27:     db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:28:     db.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_harness.py:50:     db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:65:     db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:71:     db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:87:     db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:122:    d.add_play_segment("vid1", 0.0, 10.0)
backend\pipeline\segmenters\yolo_hybrid.py:348:         segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
tests\test_eval_regression.py:15:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:16:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_regression.py:56:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:69:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:70:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_regression.py:111:   d.add_play_segment("vid1", 0.0, 10.0)
backend\pipeline\segmenters\trajectory_hybrid.py:325:         segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\motion.py:173:         segment_id =
self.db.add_play_segment(video_id, start, end, server, receiver, metadata)
backend\pipeline\segmenters\gemini.py:458:         segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
tests\test_fusion_hybrid.py:45:    db.add_play_segment.return_value = 200
tests\test_gemini.py:32:    db_mock.add_play_segment.return_value = "segment_1"
tests\test_gemini.py:71:    db_mock.add_play_segment.side_effect = ["segment_1", "segment_2"]
tests\test_gemini.py:105:    db_mock.add_play_segment.return_value = "segment_1"
tests\test_job_worker_loop.py:71:        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
tests\test_job_worker_loop.py:139:        sid = self.db.add_play_segment(video_id, 0.0, 5.0)
tests\test_reingest_safety.py:12:    sid = db.add_play_segment(vid, 1.0, 4.0)
tests\test_reingest_safety.py:30:    db.add_play_segment("vid1", 1.0, 2.0)           # old
tests\test_reingest_safety.py:33:    db.add_play_segment("vid1", 10.0, 12.0)         # new run
tests\test_reingest_safety.py:45:    db.add_play_segment("vid1", 1.0, 2.0)           # prior
good result
tests\test_tracknet_hybrid.py:60:    db.add_play_segment.return_value = 200
tests\test_tracknet_hybrid.py:126:    db.add_play_segment.return_value = 200
tests\test_tracknet_hybrid.py:150:    db.add_play_segment.return_value = 200
tests\test_tracknet_hybrid.py:178:    db.add_play_segment.side_effect = [201, 202]
tests\test_tracknet_hybrid.py:192:    meta = db.add_play_segment.call_args_list[0].args[5]
tests\test_trajectory_hybrid_equivalence.py:46:    db.add_play_segment.return_value = 200
tests\test_yolo_hybrid.py:70:    db_mock.add_play_segment.return_value = "yolo_segment_1"
tests\test_yolo_hybrid.py:138:    db_mock.add_play_segment.return_value = "segment_ok"
tests\test_yolo_hybrid.py:207:    db_mock.add_play_segment.return_value = 101
tests\test_yolo_hybrid.py:257:    db_mock.add_play_segment.return_value = 101

```

25. assistant (2026-06-18T07:53:38.678Z)

Let me check the Gemini segmenter path ? this is the pure-AI path where Gemini returns rally timestamps. Gemini's output is not subject to the velocity/dead-time-merge windowing; it's whatever the LLM returns, validated/clamped/deduped. This is a real production path (Gemini is "the cloud AI"). Let me read the gemini validation/dedup logic.

26. user (2026-06-18T07:53:40.171Z)

```

400         if curr["start"] < prev["end"]:
401             prev_dur = prev["end"] - prev["start"]
402             curr_dur = curr["end"] - curr["start"]
403             kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404             kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"

```

```

405         dropped_label = f"Segment #{dropped['idx']+1}
[{'dropped['start']:.2f}s - {'dropped['end']:.2f}s]"
406         rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
407         if kept == curr:
408             non_overlapping[-1] = curr # Replace previous with current
409         else:
410             non_overlapping.append(curr)
411         validated = non_overlapping
412
413     # Log validation summary
414     if rejected:
415         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
416         logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
"\n".join(reject_lines))
417         logger.info(f"{len(validated)} play segments passed validation (out of
{len(ai_segments)} from Gemini).")
418
419         stitch_cfg = self.config.get("stitching", {})
420         begin_pad = stitch_cfg.get("begin_padding", 0.5)
421         end_pad = stitch_cfg.get("end_padding", 0.5)
422
423         total_highlight_dur = 0.0
424         for c in validated:
425             total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
426
427         mins = int(total_highlight_dur // 60)
428         secs = int(total_highlight_dur % 60)
429         logger.info(
430             f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
431             f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
{end_pad}s end padding per segment)"
432         )
433
434     # 9. Populate SQLite DB with validated play segments
435     for c in validated:
436         start = c["start"]
437         end = c["end"]
438         segment = c["raw"]
439         dur = end - start
440
441         # Extract shot_count from Gemini or compute estimate if not present
442         shot_count = segment.get("shuttle_exchange_count")
443         if shot_count is None:
444             shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
445         else:
446             try:
447                 shot_count = int(shot_count)
448             except Exception:
449                 shot_count = max(3, int(dur / 0.8))
450
451         # Only store metrics we actually know ? no fabricated data
452         metadata = {
453             "shot_count": shot_count,
454             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
455             "detector": model_name
456         }
457
458         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
459
460         if segment_id:
461             segments_data.append({

```

```

462             "id": segment_id,
463             "start_time": start,
464             "end_time": end,
465             "duration": dur,
466             "metadata": metadata
467         })
468
469     return segments_data

```

27. assistant (2026-06-18T07:53:46.448Z)

This is the key reachability path. The Gemini segmenter:

1. Validates segments by removing **overlapping** ones (line 400: ``if curr["start"] < prev["end"]`` ? only true overlap), keeping non-overlapping ones.
2. There is **NO** inter-segment gap enforcement ? two Gemini-returned rallies that are merely adjacent (e.g. rally A ends at 30.0, rally B starts at 30.4, a 0.4s gap) both pass validation as "non-overlapping" and both get persisted via ``add_play_segment``.

Then in ``compile_highlights``, ``_merge_overlapping`` keeps them distinct (0.4s gap, no overlap), and padding (begin 0.5 + end 1.0 = 1.5s default in the compiler) makes the clips overlap: clip A ends at 30.0+1.0=31.0, clip B starts at 30.4-0.5=29.9 ? frames 29.9s?31.0s replay twice.

Let me confirm the validation only checks for overlap (no min-gap merge) by reading the full validation block above line 400.

28. user (2026-06-18T07:53:47.912Z)

```

360         validated = []
361         rejected = []
362
363         for c in candidates:
364             label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366             # A: Reject segments that start beyond the video
367             if video_duration > 0 and c["start"] >= video_duration:
368                 rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369                 continue
370
371             # B: Clamp end_time to video duration
372             if video_duration > 0 and c["end"] > video_duration:
373                 logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374                 c["end"] = video_duration
375
376             dur = c["end"] - c["start"]
377
378             # C: Reject segments shorter than min_segment_duration
379             if dur < min_segment_duration:
380                 rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381                 continue
382
383             # D: Flag suspiciously long segments
384             if dur > max_segment_duration:
385                 logger.warning(
386                     f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
387                     f"This may be a detection error. Keeping but flagging."
388                 )
389
390             validated.append(c)
391
392             # E: Resolve overlapping segments (keep the longer one)
393             if len(validated) > 1:

```

```

394         # Sort by start time
395         validated.sort(key=lambda r: r["start"])
396         non_overlapping = [validated[0]]
397         for curr in validated[1:]:
398             prev = non_overlapping[-1]
399             # Check if current segment overlaps with the previous one
400             if curr["start"] < prev["end"]:
401                 prev_dur = prev["end"] - prev["start"]

```

29. assistant (2026-06-18T07:53:54.080Z)

Confirmed. The Gemini validation only deduplicates **strictly overlapping** segments (`curr["start"] < prev["end"]`). Two non-overlapping but close rallies (gap < 1.5s) both survive and both get persisted to the DB. They then reach `compile_highlights` via the `/api/compile` endpoint (or the script path), and the per-clip padding makes their output clips overlap.

This is a genuine, reachable production path. The Gemini segmenter is the named cloud AI provider, the `/api/compile` endpoint reads exactly these persisted segments, and there is no gap enforcement anywhere between Gemini output and the per-clip padding.

Let me double-check the compiler's padding defaults vs. what the API wires in. The compiler constructor uses `end_padding=1.0, begin_padding=0.5`. The API passes `stitching.begin_padding` / `stitching.end_padding`. Let me confirm the config defaults so the combined padding magnitude is concrete.

30. user (2026-06-18T07:53:55.637Z)

```

240:     begin_padding: float = 0.5
241:     end_padding: float = 0.5

```

31. user (2026-06-18T07:53:56.550Z)

```

73:         "begin_padding": 0.5,
74:         "end_padding": 0.5,

```

32. assistant (2026-06-18T07:54:09.146Z)

The configured defaults are `begin_padding=0.5` and `end_padding=0.5` (combined 1.0s). So via the API/config path, the combined padding is 1.0s. Any two Gemini-returned rallies separated by a raw gap < 1.0s will overlap after padding. (Via the compiler's own constructor defaults ? used in the `test_compiler.py` no-config case ? `end_padding=1.0` makes the threshold 1.5s, but the production wiring uses the config's 1.0s combined.)

A gap < 1.0s between two distinct rallies in Gemini output is entirely plausible (a quick reserve after a let or fault, two short rallies back-to-back). So the overlap is reachable, though the magnitude is modest (sub-1.0s of duplicated footage at the seam).

Let me verify there's no second merge pass on padded ranges anywhere, and confirm `_merge_overlapping` is the only collapse ? which I already saw (it operates on raw, unpadded ranges only). The reachability is established.

Now let me assess the lens question precisely:

- **real**: Yes, the logic defect is real ? padding is applied per-clip after the raw-range merge, so adjacent-but-distinct ranges produce overlapping output clips. The module's stated invariant ("never replay the same footage") is violated at the seam.
- **reachable**: Yes ? the Gemini provider path persists non-overlapping-but-close segments (gap < combined padding) with no gap enforcement; `/api/compile` feeds them straight into the compiler. Also the default-OFF detector cap/blob-split features create adjacent windows, but those are flag-gated. The Gemini path is default-reachable.
- **already_guarded_or_tested**: No test asserts non-overlap of padded output across adjacent clips; `test_merge_overlapping` line 28 actually *confirms* gapped rallies stay split (the precondition for the overlap), and no test checks the padded seams.

This is a low-severity correctness/quality defect (duplicated footage at clip seams), not a

crash. The reachability is real but the trigger is a specific input shape (two rallies closer than the combined padding). I'll set real=true, reachable=true.

33. user (2026-06-18T07:54:25.765Z)

Structured output provided successfully

34. assistant (2026-06-18T07:54:35.001Z)

I've completed the reachability analysis and submitted my findings.

Summary: The candidate bug is **real and reachable** (low severity).

The defect in ``backend/pipeline/stitcher/compiler.py`` (lines 201 vs 268-269) is genuine: ``_merge_overlapping`` collapses only raw, unpadded ranges, then padding is applied independently per clip, so two distinct rallies closer than the combined padding produce overlapping output clips ? violating the module's own "never replay the same footage" invariant.

Key reachability finding: the candidate's proposed trigger (the standard detector path) is actually NOT reachable on defaults, because the motion/hybrid segmenters merge windows within ``dead_time_merge_gap=2.0s`` (``backend/pipeline/segmenters/motion.py:146``), guaranteeing persisted segments are ?2.0s apart ? exceeding the 1.0s combined config padding. However, I found a **different default-ON path that does reach it**: the Gemini provider (``backend/pipeline/segmenters/gemini.py:400``) deduplicates only **strictly overlapping** segments with no inter-segment gap enforcement, persists close-but-distinct rallies via ``add_play_segment``, and ``/api/compile`` (``backend/main.py:573-623``) feeds them straight into the padded compiler. Two Gemini rallies with a raw gap < 1.0s overlap after padding.

The flag-gated ``window_hard_cap`` / ``window_blob_fallback`` cap-splitters in ``tracknet_runner.py`` are an even stronger (adjacent-window) source but default-OFF.

No existing test guards the padded seam.